Statyczne elementy członkowskie klas

Lekcja 13 - Programowanie aplikacji

Klasa 4 technik programista | Kwalifikacja INF.04

```
class Pracownik {
public:
  static string s_szkola;
  string imie;
};

Pracownik::s_szkola =
  "Technikum";
```

static = jeden wspólny element klasy

1. Po co są elementy statyczne?

- Zwykłe pola obiektu opisują konkretną instancję, np. imię jednego ucznia.
- Element statyczny należy do klasy jako całości, a nie do pojedynczego obiektu.
- Dzięki temu jedna informacja może być wspólna dla wszystkich obiektów danej klasy.
- W C++ deklarujemy go słowem kluczowym static.

Przykład myślowy

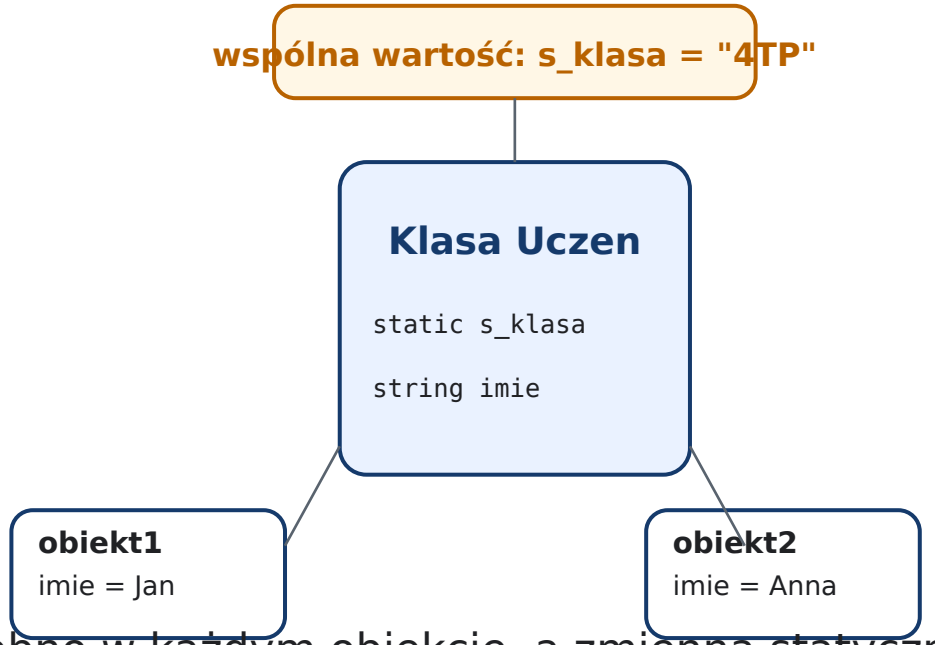
Jeżeli wszyscy uczniowie należą do klasy 4TP, nie trzeba przechowywać tej wartości osobno w każdym obiekcie.

2. Nowe pojęcia i skróty

Pojęcie	Znaczenie
OOP	Object-Oriented Programming - programowanie obiektowe, czyli budowanie programu z klas i obiektów.
static	Słowo kluczowe oznaczające element wspólny dla klasy.
instancja	Konkretny obiekt utworzony na podstawie klasy.
operator zakresu ::	Pozwala odwołać się do elementu klasy, np. Pracownik::s_szkola.
akcesor	Metoda udostępniająca kontrolowany dostęp do pola, np. getter lub setter.

3. Zmienna instancyjna a statyczna

Cecha	zmienna instancyjna	zmienna statyczna
Liczba kopii	osobna dla obiektu	jedna dla całej klasy
Dostęp	przez obiekt	najlepiej przez klasę
Przykład	pracownik1.imie	Pracownik::s_szkola



Najważniejsza różnica: zmienna instancyjna istnieje osobno w każdym obiekcie, a zmienna statyczna ma jedną wspólną wartość dla klasy.

4. Deklaracja i inicjalizacja zmiennej statycznej

Schemat zapisu w C++

```
class Pracownik {
public:
    static string s_szkola;     // deklaracja w klasie
    string imie;                // pole instancyjne
};

string Pracownik::s_szkola = "Technikum Informatyczne";
// inicjalizacja poza klasą, w zakresie globalnym
```

- static w klasie oznacza deklarację elementu wspólnego.
- Zmienną statyczną zwykle trzeba zainicjalizować poza definicją klasy.
- Do odwołania używamy nazwy klasy i operatora ::.
- Wyjątki dotyczą m.in. niektórych stałych const typów całkowitych.

5. Dostęp przez klasę jest czytelniejszy

Odwołanie do elementu statycznego

```
Pracownik pracownik1;
pracownik1.imie = "Jan";

// zalecany dostęp do elementu statycznego:
Pracownik::s_szkola = "Technikum";

// możliwe, ale mniej czytelne:
pracownik1.s_szkola = "Technikum";
```

Wniosek

Skoro element statyczny należy do klasy, zapis Pracownik::s_szkola lepiej pokazuje intencję programu.

Uwaga

Zmiana wartości pola statycznego jest widoczna dla wszystkich obiektów tej klasy.

6. Przykład 11.6 - publiczne zmienne klasowe

Fragment klasy z przykładu

```
class Pracownik {
public:
    static string s_szkola;
    static string s_stanowisko;
    string imie;
    string nazwisko;
    void wyswietlDane();
};

string Pracownik::s_szkola = "Technikum Informatyczne";
string Pracownik::s_stanowisko = "nauczyciel";
```

- s_szkola i s_stanowisko są wspólne dla wszystkich pracowników.
- imie i nazwisko są osobne dla każdego obiektu.
- Po zmianie Pracownik::s_stanowisko nowa wartość dotyczy kolejnych odczytów w całej klasie.
- Przykład pokazuje różnicę między danymi klasy i danymi obiektu.

7. Statyczne funkcje członkowskie

- Statyczna funkcja członkowska, czyli metoda statyczna, także jest oznaczona słowem static.
- Można ją wywołać bez tworzenia obiektu, np. Pracownik::s_zwrocSzkola().
- Może korzystać ze statycznych pól i wywoływać inne statyczne metody klasy.
- Nie ma bezpośredniego dostępu do pól instancyjnych, bo nie działa na konkretnym obiekcie.

Prototypy metod statycznych

```
static string s_zwrocSzkola();
static void s_ustawSzkola(string pSzkola);
```

8. Przykład 11.7 - prywatna zmienna klasowa i akcesory

Akcesory do prywatnego pola statycznego

```
class Pracownik {
    static string s_szkola;   // private - ukryta w klasie
public:
    string imie;
    string nazwisko;
    static void s_ustawSzkola(string pSzkola);
    static string s_zwrocSzkola();
};

void Pracownik::s_ustawSzkola(string pSzkola) {
    s_szkola = pSzkola;
}
string Pracownik::s_zwrocSzkola() {
    return s_szkola;
}
```

- Pole statyczne może być prywatne, tak jak zwykłe pole klasy.
- Dostęp z zewnątrz realizują publiczne metody statyczne.
- Setter ustawia wartość wspólną, a getter ją zwraca.
- To bezpieczniejsze niż publiczna zmienna klasowa.

9. Czego metoda statyczna nie może robić?

Typowy błąd

```
class Uczen {
    static string s_klasa;
    string imie;
public:
    static void pokaz() {
        cout << s_klasa;  // OK
        cout << imie;     // BŁĄD: brak konkretnego
obiektu
    }
};
```

Dlaczego?

Metoda statyczna nie ma wskaźnika this do konkretnej instancji. Nie wie, którego obiektu imię miałaby odczytać.

10. Kiedy stosować static?

- Gdy informacja jest wspólna dla wszystkich obiektów klasy, np. nazwa szkoły, licznik obiektów, wspólna konfiguracja.
- Gdy metoda wykonuje operację związaną z klasą, ale nie potrzebuje danych konkretnego obiektu.
- Gdy chcemy przygotować kontrolowany dostęp do prywatnej zmiennej klasowej.
- Nie używamy static tylko po to, aby „łatwiej dostać się do danych” z dowolnego miejsca programu.

11. Mini-zadanie na lekcji

Zaprojektuj klasę Uczen:

- pola instancyjne: imie, nazwisko, numer_w_dzienniku;
- statyczne pola prywatne: s_klasa, s_zawod;
- statyczne metody: ustawKlase(), zwrocKlase(), ustawZawod(), zwrocZawod();
- metoda instancyjna wyswietlDane(), która pokaże dane ucznia i wspólne dane klasy.

Warunek

W programie utwórz dwa obiekty Uczen i pokaż, że zmiana wartości statycznej jest widoczna dla obu obiektów.

12. Najczęstsze błędy

- Brak inicjalizacji zmiennej statycznej poza klasą.
- Mylenie pola wspólnego dla klasy z polem osobnym dla obiektu.
- Odwoływanie się w metodzie statycznej do pola instancyjnego bez obiektu.
- Nadużywanie publicznych zmiennych statycznych zamiast prywatnych pól i akcesorów.
- Próba inicjalizowania zwykłych pól tak, jak pól statycznych.

13. Podsumowanie

- Element statyczny należy do klasy, a nie do pojedynczego obiektu.
- Zmienna statyczna ma jedną wspólną wartość dla wszystkich instancji klasy.
- Do statycznych elementów najlepiej odwoływać się przez nazwę klasy i operator ::.
- Metoda statyczna może korzystać ze statycznych danych, ale nie z danych konkretnego obiektu bez jego wskazania.
- Prywatne zmienne statyczne warto udostępniać przez statyczne metody dostępowe.